Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE1113 Sistemas Empotrados
Prof. Dr.-Ing. Jeferson González Gómez

Taller 6: Desarrollo de Firmware con QEMU y RISC-V

Emulación de Hardware y Firmware Bare-Metal

1. Introducción

Este taller introduce los conceptos fundamentales para desarrollar firmware bare-metal (sin sistema operativo) en una arquitectura RISC-V emulada con QEMU. A diferencia de los talleres anteriores que se enfocaban en Linux embebido con Yocto, este taller le permite trabajar directamente con el hardware emulado, escribir código de inicialización en ensamblador RISC-V y construir software de bajo nivel sobre interfaces MMIO reales.

QEMU en el desarrollo moderno de sistemas empotrados

El desarrollo de sistemas empotrados modernos se enfrenta a un reto fundamental: el software y el hardware evolucionan en paralelo, pero el hardware físico rara vez está disponible desde el inicio del proyecto. En este contexto, la emulación precisa de plataformas juega un papel central en las metodologías actuales de co-diseño HW/SW (*hardware/software co-design*), permitiendo que los equipos de firmware avancen de forma independiente al equipo de hardware [1].
QEMU (*Quick Emulator*) es un emulador y virtualizador de código abierto que modela plataformas completas con alta fidelidad: procesador, memoria, controladores de interrupción y periféricos de entrada/salida mapeados a memoria (Memory-Mapped I/O - MMIO). Su arquitectura modular permite extender la máquina virtual con dispositivos personalizados escritos en C, lo que lo convierte en una herramienta estándar en la industria para validación temprana de firmware, pruebas de regresión automatizadas y educación en sistemas empotrados [1, 2].
En este taller se utiliza una versión extendida de QEMU (fork `qemu-tec`) que incorpora un dispositivo MMIO educativo (`riscv-edu-timer`) integrado directamente en la máquina virtual `virt` de RISC-V. Este enfoque refleja el flujo real de trabajo en la industria: el equipo de arquitectura de hardware define la interfaz del periférico (registros, semántica, interrupciones) y el equipo de firmware la implementa y valida sobre el modelo, antes de que exista silicio.
El objetivo del taller es que el estudiante comprenda cómo funciona un sistema embebido desde las capas más bajas: inicialización de memoria, acceso a hardware mediante memory-mapped I/O (MMIO), configuración de periféricos y validación del comportamiento del firmware. Al finalizar, el estudiante será capaz de manipular el timer configurable desde software/firmware, crear perfiles de operación y verificar su efecto sin modificar el hardware emulado.

2. Inicio Rápido

El entorno de desarrollo corre completamente dentro de Docker: no es necesario instalar QEMU ni el toolchain RISC-V en el sistema operativo del estudiante. El único requisito es tener Docker instalado.
La estructura del repositorio es la siguiente:

```
taller06-firmware-qemu/
  Dockerfile                  # Imagen con QEMU + toolchain RISC-V
  src/kernel-minimal/         # Código fuente del firmware
    app.c                     # <-- Archivo principal a modificar
    devices.c / devices.h     # Driver MMIO del UART y Timer
    main.c  start.S  linker.ld
  scripts/
    build.sh                  # Compila el firmware
    run.sh                    # Ejecuta en QEMU
```

Paso 1: Construir la imagen Docker

```
docker build --build-arg UID=$(id -u) --build-arg GID=$(id -g) \
             -t taller06 .
```

Este comando clona el fork de QEMU (`qemu-tec`), compila el binario `qemu-system-riscv32` con el device `riscv-edu-timer` integrado e instala el toolchain RISC-V dentro de la imagen. Tarda aproximadamente 10–15 minutos y solo es necesario ejecutarlo una vez.

Paso 2: Iniciar el contenedor

```
docker run -it --rm -v "$(pwd)":/workspace taller06
```

El directorio del repositorio queda montado en `/workspace` dentro del contenedor. Los comandos siguientes se ejecutan desde ahí.

Paso 3: Compilar y ejecutar el firmware

```
./scripts/build.sh   # compila src/kernel-minimal/ -> build/firmware.
   elf
./scripts/run.sh     # carga el firmware en QEMU
```

La salida serial del firmware aparece en la terminal. Para salir de QEMU: `Ctrl+A` seguido de `X`.

Ciclo de desarrollo: editar `src/kernel-minimal/app.c`, luego ejecutar `build.sh` y `run.sh` sin necesidad de reiniciar el contenedor ni reconstruir la imagen.

3. Conceptos Fundamentales

3.1. Bare-Metal vs. Linux Embebido

En los talleres anteriores trabajamos con Linux embebido, donde el kernel gestiona recursos de hardware, memoria y proporciona servicios del sistema operativo. En cambio, en un entorno bare-metal el firmware tiene acceso directo al hardware sin intermediarios. Esto significa que el programador es completamente responsable de la inicialización del sistema: configuración de memoria, stack, limpieza de secciones no inicializadas, etc.
El firmware bare-metal es especialmente importante en sistemas embebidos restrictivos: microcontroladores con pocos kilobytes de RAM, dispositivos que requieren latencia predecible, o sistemas críticos donde la sobrecarga de un kernel resulta inaceptable.

3.2. RISC-V y su Arquitectura

RISC-V (Reduced Instruction Set Computer) es una arquitectura de conjunto de instrucciones abierta y libre, diseñada de forma modular. En este taller utilizamos RV32I (RISC-V 32-bit Integer), la extensión base más simple, que es suficiente para aprender los principios fundamentales.
La arquitectura RISC-V define 32 registros de propósito general: `x0` (hardwired a cero), `x1` (link register), `x2` (stack pointer), y `x3` a `x31`. El Application Binary Interface (ABI) usado en este taller, ilp32, define convenciones para pasar argumentos a funciones, preservar registros y alinear datos en memoria.

3.3. Memory-Mapped I/O (MMIO)

Los dispositivos en un sistema embebido se acceden típicamente mediante MMIO: registros de hardware mapeados a direcciones de memoria. En lugar de usar instrucciones especiales `in` y `out` (como en x86), RISC-V trata los registros de dispositivos como posiciones de memoria ordinarias. Esto es conceptualmente más simple pero requiere cuidado: los accesos a dispositivos no se pueden optimizar ni reordenar como con memoria ordinaria, por lo que se usan punteros `volatile`.

3.4. QEMU como Herramienta de emulación

QEMU es un emulador de sistema completo que simula una máquina RISC-V virtual. Permite ejecutar firmware sin hardware real y correr ciclos rapidos de implementación, prueba y ajuste sobre dispositivos MMIO. La imagen Docker del taller incluye una versión de QEMU con el device `riscv-edu-timer` integrado directamente en la máquina `virt`; el estudiante interactúa con él a través de MMIO sin necesidad de conocer su implementación interna.

4. Estructura del Firmware

El firmware proporcionado consta de los siguientes archivos en `src/kernel-minimal/`:

Archivo	Rol
`start.S`	Inicialización en ensamblador: stack, BSS, copia de datos, salto a `main()`
`linker.ld`	Layout de memoria: ROM en `0x80000000`, RAM en `0x80040000`
`main.c`	Punto de entrada C: inicializa el timer y llama a `app_main()`
`devices.h`	Definiciones de direcciones MMIO, mascaras de bits y prototipos
`devices.c`	Implementación de funciones de acceso al UART y al Timer
`app.c`	**Lógica de la aplicación: el estudiante modifica este archivo**

Los estudiantes trabajarán principalmente en `app.c`, donde se implementan los perfiles de configuración del timer. Los archivos `devices.h` y `devices.c` proporcionan la capa de abstracción sobre MMIO y no deben modificarse salvo que sea necesario agregar funciones auxiliares.

4.1. Inicialización: start.S

Cuando QEMU inicia, el procesador salta a la dirección `0x80000000` donde comienza el firmware. En ese punto, el stack no está configurado, el segmento de datos no está inicializado, y no podemos ejecutar C normal. El archivo `start.S` se encarga de preparar el entorno:

```
.section .text.start
.globl _start

_start:
    # Configurar el stack pointer al final de la RAM
    la sp, _stack_end

    # Copiar datos inicializados desde ROM a RAM
    la a0, _data_lma
    la a1, _data_vma
    la a2, _edata
    ...

    # Limpiar sección BSS (variables no inicializadas)
    la a0, _bss_start
    la a1, _bss_end
    ...

    # Saltar a main
    call main
```

La instrucción `la` (load address) carga una dirección global de 32-bit. En RISC-V 32-bit, las direcciones se cargan en dos pasos: primero se carga la parte alta con `lui`, luego se ajusta con `addi`. El assembler maneja esto automáticamente.

El stack pointer (registro `sp`, alias para `x2`) se configura al final de la RAM. En arquitecturas que crecen hacia abajo (como RISC-V), el stack crece de direcciones altas a bajas, por lo que el puntero inicial debe estar arriba.

4.2. Programa Principal: main.c

Una vez que `start.S` ha configurado el entorno, se ejecuta `main()`, donde la lógica normal de C funciona. El firmware accede a los dispositivos mediante directivas MMIO, utilizando direcciones definidas en `devices.h`:

```
#include "devices.h"

int main(void) {
    timer_reset();
    timer_start_free_run(0u); /* 1 tick = 1 us */

    app_main();

    while (1) {
        __asm__ volatile("wfi");
    }

    return 0;
}
```

El calificador `volatile` indica al compilador que no debe optimizar accesos a estas direcciones: cada lectura y escritura es real, no especulativa. Sin `volatile`, el compilador podría asumir que dos lecturas de la misma dirección devuelven el mismo valor (falso en un registro de dispositivo cambiante).

4.3. Layout de Memoria: linker.ld

El script de enlazador define cómo se organizan secciones de código y datos en memoria:

```
MEMORY {
    rom (rx)  : ORIGIN = 0x80000000, LENGTH = 256K
    ram (rwx) : ORIGIN = 0x80040000, LENGTH = 256K
}

SECTIONS {
    .text : {
        *(.text.start)
        *(.text)
    } > rom

    .data : {
        *(.data)
    } > ram AT > rom

    .bss : {
        *(.bss)
```

```
    } > ram
}
```

Este layout es típico: código ejecutable (.text) en ROM (0x80000000), datos inicializados (.data) primero en ROM para poder cargarlos, pero con dirección virtual en RAM. Variables no inicializadas (.bss) van directamente en RAM. El script de enlazador define símbolos especiales como `_data_lma`, `_data_vma` que `start.S` usa para inicializar datos.

5. Memory Map de Dispositivos QEMU

La máquina virtual QEMU `virt` para RISC-V 32-bit implementa un conjunto de dispositivos en direcciones de memoria predefinidas. Conocer estas direcciones es fundamental para escribir drivers de hardware correctos.

Dispositivo	Dirección Base	descripción
UART	0x10000000	Terminal serial para E/S
Timer	0x10010000	Temporizador configurable por firmware

Estos valores se definen en el archivo `devices.h` del firmware. Al escribir drivers, es crítico verificar que las direcciones utilizadas coincidan con la documentación del hardware; un error de direccionamiento puede causar que el firmware intente acceder a un dispositivo incorrecto o inexistente, resultando en comportamiento anómalo.

6. Dispositivos Implementados

6.1. UART: comunicación Serial

El UART disponible en la máquina `virt` de QEMU es un 16550a mapeado en `0x10000000`. Para este taller se usa de forma simplificada: escribir un byte en el registro de transmisión (offset `0x00`) envía el carácter directamente a `stdout`, lo que es suficiente para reportar resultados por la terminal.

```
void uart_putchar(char c) {
    volatile uint32_t *tx = (volatile uint32_t *)UART_BASE;
    *tx = (uint32_t)c;
}
```

Las funciones `uart_puts`, `uart_putdec` y `uart_puthex` en `devices.c` se construyen sobre `uart_putchar` y permiten imprimir cadenas y valores numéricos sin depender de la biblioteca estándar C. El estudiante las usará para reportar el comportamiento del timer en cada perfil.

6.2. Timer: Temporización y configuración por firmware

El timer del taller es un dispositivo MMIO configurable desde software, pensado para práctica de firmware bare-metal. Permite ajustar período, prescaler y modo de operación sin modificar el hardware emulado.
El Timer se mapea en la dirección base `0x10010000` (revisar `devices.h`):

Offset	Nombre	Acceso	descripción
0x00	CONTROL	RW	Habilita conteo, auto-reload y reset
0x04	PRESCALER	RW	Divisor de tick: `tick = (prescaler+1) * 1us`
0x08	RELOAD	RW	Valor objetivo para expiración
0x0C	COUNTER	RW	Contador actual
0x10	STATUS	RW1C	Estado del timer (bit `EXPIRED`)
0x14	IRQ_ACK	W1C	Alias para limpiar expiración

Bits del registro `CONTROL`:

Bit	Nombre	Función
0	ENABLE	Inicia o detiene el conteo
1	AUTO_RELOAD	Reinicia `COUNTER` a 0 cuando expira
2	IRQ_EN	Reservado para extensión con IRQ real
3	RESET	Limpia contador/estado (auto-clear)

El período de expiración del timer se calcula como:

$$T_{\text{exp}} = \text{RELOAD} \times (\text{PRESCALER} + 1) \times 1\,\mu\text{s}$$

Por ejemplo, con `PRESCALER=999` y `RELOAD=500`: $T = 500 \times 1000\,\mu\text{s} = 500\,\text{ms}$.
Para configurar el timer desde firmware se usan las funciones de `devices.c`, que encapsulan los accesos MMIO:

```
/* Perfil periódico: expira cada 500 ms, reinicia automáticamente */
timer_reset();
timer_start_periodic(500u, 999u);   /* reload=500, prescaler=999 */

uint32_t count = 0;
while (count < 5) {
    while ((timer_get_status() & TIMER_STATUS_EXPIRED) == 0u) { }
    timer_ack_expired();
    count++;
    uart_puts("Expiración␣#");
    uart_putdec(count);
    uart_puts("\n");
}
```

También es posible acceder directamente a los registros mediante `devices.h` si se requiere control fino. La capa de funciones en `devices.c` es un punto de partida, no una restricción.

7. Flujo de Ejecución Detallado

Cuando se ejecuta `./scripts/run.sh`, ocurre lo siguiente:

1. QEMU inicia y carga el archivo `build/firmware.elf` en memoria en la dirección `0x80000000`.
2. El procesador salta a `0x80000000`, comenzando ejecución en `_start`.
3. `start.S` inicializa el stack, copia datos, limpia BSS, y llama a `main()`.
4. `main()` escribe mensajes al UART, accede al Timer, y entra en un loop.
5. El simulador intercepta accesos MMIO: escrituras al UART se envían a stdout, lecturas del Timer devuelven valores simulados.
6. El firmware se valida observando la salida serial y el comportamiento del timer según la configuración aplicada.

8. Validación de Firmware Bare-Metal

En este taller la validación se centra en comprobar comportamiento observable del firmware sobre MMIO.

8.1. Estrategia de validación

1. Configure el timer con una combinación de `PRESCALER`, `RELOAD` y bits de `CONTROL`.
2. Ejecute firmware y observe la salida por UART.
3. Verifique periodicidad, expiración y limpieza de estado por `STATUS/IRQ_ACK`.
4. Cambie una sola variable de configuración por iteración para aislar su efecto.

8.2. Criterios de comprobación sugeridos

- El timer inicia y se detiene cuando cambia `ENABLE`.
- El período observable cambia al modificar `PRESCALER` o `RELOAD`.
- En modo `AUTO_RELOAD`, el firmware reconoce expiraciones recurrentes.
- La limpieza de expiración (W1C) evita lecturas falsas en iteraciones siguientes.

8.3. Buenas prácticas de implementación

- Definir constantes para offsets y bits de control.
- Encapsular configuraciones en funciones de alto nivel (por ejemplo, perfiles de timer).
- Documentar supuestos de tiempo (ticks, unidad base, escala usada en reportes).

9. Evaluación

Toda la implementación se realiza modificando **únicamente archivos de firmware** (`app.c` y, si se requiere, `devices.h`/`devices.c`). No se permite modificar el código del dispositivo QEMU ni el hardware emulado.

9.1. Parte 1: Tres perfiles de temporización (40 %)

Implementar en `app_main()` exactamente tres perfiles de timer. Cada perfil debe:

1. Imprimir por UART el nombre del perfil y los valores de `PRESCALER`, `RELOAD` y `CONTROL` que se van a aplicar.
2. Configurar el timer llamando a funciones de `devices.c` o accediendo directamente a registros MMIO.
3. Esperar al menos cinco expiraciones del timer, imprimiendo el número de expiración y el valor del contador en cada una.
4. Limpiar `STATUS` mediante `IRQ_ACK` después de cada expiración.
5. Imprimir el período medido (diferencia de contador entre expiraciones consecutivas) y compararlo con el período teórico calculado con la fórmula $T = \text{RELOAD} \times (\text{PRESCALER} + 1) \times 1\,\mu\text{s}$.

Los tres perfiles deben diferenciarse de forma observable en la salida serial. Se sugiere (pero no se impone) la siguiente combinación:

Perfil	**PRESCALER**	**RELOAD**	**AUTO_RELOAD**	**Período teórico**
Rápido	0	10 000	Sí	10 ms
Lento	999	500	Sí	500 ms
One-shot	0	50 000	No	50 ms (una vez)

El perfil one-shot debe demostrar que el timer se detiene tras la primera expiración (bit `ENABLE` se limpia automáticamente) y que una segúnda lectura de `STATUS` no reporta nueva expiración sin reconfigurar.

Ejemplo de salida esperada para el perfil rápido:

```
--- Perfil Rápido: PRESCALER=0 RELOAD=10000 AUTO_RELOAD=1 ---
T_teórico = 10000 us
Exp #1: contador=10001  delta=10001 us
Exp #2: contador=10000  delta=10000 us
Exp #3: contador=10001  delta=10001 us
Exp #4: contador=10000  delta=10000 us
Exp #5: contador=10000  delta=10000 us
```

9.2. Parte 2: Abstracción de software sobre el timer (20 %)

Implementar **una** de las siguientes opciones. La elección debe indicarse en el README.

Opción A — Estructura de perfil y API de aplicación.

Definir una estructura `TimerProfile_t` con campos `prescaler`, `reload` y `auto_reload`, y una función `timer_apply_profile(const TimerProfile_t *p)`. Reimplementar los tres perfiles de la Parte 1 declarando instancias de la estructura y aplicandolas con esa función. Demostrar que agregar un cuarto perfil requiere solo una nueva instancia de la estructura, sin cambiar la lógica de configuración.

Opción B — Selector de perfil en tiempo de compilación.

Usar una macro `TIMER_PROFILE` (valor 0, 1 o 2) que determine que perfil se compila. En `app.c`, usar `#if/#elif` para incluir la configuración correspondiente. Demostrar con tres compilaciones distintas (`make CFLAGS=-DTIMER_PROFILE=0`, `=1`, `=2`) que cada una produce salida diferente en QEMU.

Opción C — Conmutación de perfiles en ejecución.

El firmware ejecuta los tres perfiles en secuencia dentro de un bucle: tras completar las cinco expiraciones de un perfil, imprime un mensaje de transición y configura el siguiente. Al terminar el tercero, vuelve al primero. El programa debe ejecutar al menos dos ciclos completos y terminar limpiamente.

9.3. Parte 3: Reloj digital bare-metal (40 %)

Implementar un reloj digital que use el timer para avanzar el tiempo segundo a segundo y muestre la hora actual por UART con el formato `HH:MM:SS`.

Requisitos de configuración

Crear el archivo `clock_config.h` con la hora inicial definida mediante macros:

```
#ifndef CLOCK_CONFIG_H
#define CLOCK_CONFIG_H

#define CLOCK_INIT_HH  9
#define CLOCK_INIT_MM  30
#define CLOCK_INIT_SS  0

#endif
```

El reloj debe arrancar con los valores de ese archivo sin necesidad de modificar `app.c`.

Requisitos de comportamiento

1. Configurar el timer en modo `AUTO_RELOAD` con un período de exactamente 1 segundo. Justificar en el informe los valores de `PRESCALER` y `RELOAD` elegidos usando la fórmula $T = \text{RELOAD} \times (\text{PRESCALER} + 1) \times 1\,\mu\text{s}$.
2. En cada expiración, limpiar `STATUS` e incrementar los segundos.
3. Manejar correctamente los desbordamientos: segundos 0–59, minutos 0–59, horas 0–23.
4. Imprimir la hora actualizada por UART en cada tick de segundo:

```
09:30:00
09:30:01
09:30:02
...
```

5. El reloj debe correr de forma indefinida (bucle sin condición de salida).

Restricciones

- No se permite usar variables globales no inicializadas para el estado del reloj; declarar la hora como una estructura o tres variables locales inicializadas desde `clock_config.h`.
- No se permite usar funciones de biblioteca estándar (`printf`, `sprintf`, etc.); usar exclusivamente `uart_putchar`/`uart_puts`/`uart_putdec` de `devices.c`.

Entregables

(subir al TecDigital)

- Archivo comprimido `ApellidoNombre_Taller06.tar.gz` con:
  1. Código fuente modificado en `src/kernel-minimal/` (archivos `.c` y `.h`, incluyendo `clock_config.h`).
  2. `README.txt`: nombre, carné, opción elegida en Parte 2, instrucciones para compilar y ejecutar.
- Informe PDF (maximo 3 paginas) con:
  1. Tabla de los tres perfiles (Parte 1): `PRESCALER/RELOAD/CONTROL`, período teórico y medido.
  2. Captura de la salida serial de los tres perfiles.
  3. Descripción de la opción de Parte 2 implementada y fragmento de código relevante.
  4. Justificación de la configuración del timer para 1 segundo (Parte 3) y captura de al menos 10 segundos de salida del reloj.

Referencias

[1] F. Bellard, "QEMU, a Fast and Portable Dynamic Translator," in *Proc. USENIX Annual Technical Conference (FREENIX Track)*, pp. 41–46, 2005.

[2] The QEMU Project, *QEMU: the FAST! processor emulator* [software], version 9.x, disponible en: `https://www.qemu.org`, 2024.